API Reference

OpenAI compatibility

Copy page

Connect OpenAI clients to Ollama. Ollama supports a subset of the **OpenAI API**.

Direct cloud access

Set your **API key** in `OLLAMA_API_KEY`. Install the client with `pip install openai`. No Ollama installation required.

```python
import os
from openai import OpenAI

client = OpenAI(
    base_url="https://ollama.com/v1",
    api_key=os.environ["OLLAMA_API_KEY"],
)

response = client.chat.completions.create(
    model="gemma4:31b",
    messages=[{"role": "user", "content": "Say hello in one sentence."}],
)
print(response.choices[0].message.content)
```

The same client supports stateless Responses requests:

```
response = client.responses.create(
    model="gemma4:31b",
    input="Say hello in one sentence.",
)
print(response.output_text)
```

Choose a **cloud model**. The cloud API does not support stateful Responses, built-in web search through `/v1/responses`, or custom/freeform tool-call replay.

Local server usage

These examples connect to your local Ollama server. The client requires an API key value, but Ollama ignores it. To use cloud models through this server, **sign in to Ollama**.

Simple `/v1/chat/completions` example

basic.py — Python

```
from openai import OpenAI

client = OpenAI(
    base_url='http://localhost:11434/v1/',
    api_key='ollama',  # required but ignored
)

chat_completion = client.chat.completions.create(
    messages=[
        {
            'role': 'user',
            'content': 'Say this is a test',
        }
    ],
    model='gpt-oss:20b',
)
print(chat_completion.choices[0].message.content)
```

Simple `/v1/responses` example

responses.py Python

```python
from openai import OpenAI

client = OpenAI(
    base_url='http://localhost:11434/v1/',
    api_key='ollama',  # required but ignored
)

responses_result = client.responses.create(
  model='qwen3:8b',
  input='Write a short poem about the color blue',
)
print(responses_result.output_text)
```

`/v1/chat/completions` with vision example

vision.py

Python

```python
from openai import OpenAI

client = OpenAI(
    base_url='http://localhost:11434/v1/',
    api_key='ollama',  # required but ignored
)

response = client.chat.completions.create(
    model='qwen3-vl:8b',
    messages=[
        {
            'role': 'user',
            'content': [
                {'type': 'text', 'text': "What's in this image?"},
                {
                    'type': 'image_url',
                    'image_url': '
                },
            ],
        }
    ],
    max_tokens=300,
)
print(response.choices[0].message.content)
```

Endpoints

`/v1/chat/completions`

Supported features

- Chat completions
- Streaming
- JSON mode
- Reproducible outputs



Vision

Tools

Reasoning/thinking control (for thinking models)

- [ ] Logprobs

Supported request fields

`model`

`messages`

- Text `content`
- Image `content`
  - Base64 encoded image
  - [ ] Image URL
- Array of `content` parts

`frequency_penalty`

`presence_penalty`

`response_format`

`seed`

`stop`

`stream`

`stream_options`

- `include_usage`

`temperature`

`top_p`

`max_tokens`

`tools`

`reasoning_effort` (model-defined names and compatibility aliases)

`reasoning`

`effort` (model-defined names and compatibility aliases)

- [ ] `tool_choice`
- [ ] `logit_bias`
- [ ] `user`
- [ ] `n`

`reasoning_effort` and `reasoning.effort` control model thinking. Use `/api/show` to discover each model's supported values and default. When a model uses its metadata to resolve named levels, supported names are applied exactly and unsupported names resolve to the model default. Boolean-only models that support enabling thinking map recognized compatibility efforts to `true`, while `"none"` requests `false`. Models without metadata retain the existing compatibility aliases: `"minimal"` maps to `"low"`, and `"xhigh"` or `"ultra"` map to `"max"`. GPT-OSS keeps its existing OpenAI compatibility conversion: `"minimal"` uses `"low"`, `"xhigh"` or `"ultra"` use `"high"`, and `"none"` uses `"low"` because GPT-OSS cannot disable thinking.

`/v1/completions`

Supported features

- Completions
- Streaming
- JSON mode
- Reproducible outputs
- [ ] Logprobs

Supported request fields

- `model`
- `prompt`
- `frequency_penalty`
- `presence_penalty`



- `seed`
- `stop`
- `stream`
- `stream_options`
  - `include_usage`
- `temperature`
- `top_p`
- `max_tokens`
- `suffix`
- [ ] `best_of`
- [ ] `echo`
- [ ] `logit_bias`
- [ ] `user`
- [ ] `n`

Notes

- `prompt` currently only accepts a string

`/v1/models`

Notes

- `created` corresponds to when the model was last modified
- `owned_by` corresponds to the ollama username, defaulting to `"library"`

`/v1/models/{model}`

Notes

- `created` corresponds to when the model was last modified

- `owned_by` corresponds to the ollama username, defaulting to `"library"`

`/v1/embeddings`

Supported request fields

- `model`
- `input`
  - string
  - array of strings
  - array of tokens
  - array of token arrays
- `encoding format`
- `dimensions`
- `user`

`/v1/audio/speech`

> Note: Added in Ollama v0.40.8

Generates audio with **speech models**. The response streams while audio is generated.

Supported request fields

- `model`
- `input` (up to 4,096 characters)
- `voice` (one of the voices `ollama show` lists, in any case and with or without accents, such as `Freya`; omit it for the model's `default` voice)
- `response_format`
  - `wav` (default)
  - `pcm`

- [ ] `mp3` , `opus` , `aac` , `flac`

`speed` (`1` only)

`stream_format` (`audio` only)

- [ ] `instructions`

Notes

- Audio is 16-bit, mono, 24 kHz. `pcm` returns raw little-endian samples with no header.
- The default `response_format` is `wav` , not `mp3` .
- Unsupported values return a `400` error instead of being ignored.

`/v1/audio/transcriptions`

Transcribes speech with models that accept audio input.

Supported request fields

`file` (WAV at up to 384 kHz; up to 512 MB and 2 hours)

`model`

`language` (ISO 639-1 code such as `fr` , or an English language name; omit it to let the model detect the language)

`prompt` (used by models without speech-specific instructions)

`response_format`

`json` (default)

`text`

`verbose_json` (one segment per transcribed piece of audio; `language` is the output language for translations; for transcriptions it is set only when given in the request)

- [ ] `srt` , `vtt`

`temperature` (default `0`)

- `stream` (server-sent `transcript.text.delta` and `transcript.text.done` events)
- [ ] `timestamp_granularities`

Notes

- Audio longer than the model takes at once (30 seconds for current models) is split at pauses and transcribed piece by piece. Streaming sends each piece as it finishes.
- Each model gets the transcription instructions its publisher documents.

`/v1/audio/translations`

> Note: Added in Ollama v0.40.12

Translates speech into text in another language.

Supported request fields

- `file`
- `model`
- `to_language` (Ollama extension; ISO 639-1 code of the output language, default `en`)
- `language` (Ollama extension; the spoken language, if known)
- `prompt` (used by models without speech-specific instructions)
- `response_format`, `temperature` and `stream`, as for transcriptions

Responses API

`POST /v1/responses`

> Note: Added in Ollama v0.13.3



Ollama supports the **OpenAI Responses API**. Only the non-stateful flavor is supported (i.e., there is no `previous_response_id` or `conversation` support).

Supported features

- Streaming
- Tools (function calling)
- Reasoning summaries (for thinking models)
- [ ] Stateful requests

Supported request fields

- `model`
- `input`
- `instructions`
- `tools`
- `stream`
- `temperature`
- `top_p`
- `max_output_tokens`
- `reasoning`
  - `effort` (model-defined names and compatibility aliases)
- `think` (Ollama extension; boolean, model-defined string, or `null`)
- [ ] `previous_response_id` (stateful v1/responses not supported)
- [ ] `conversation` (stateful v1/responses not supported)
- [ ] `truncation`

Models

For local inference, download a model before using it:



```
ollama pull llama3.2
```

Direct cloud requests use the identifiers from `https://ollama.com/api/tags` and do not require a pull. Through a signed-in local server, select a cloud model such as `gemma4:cloud` .

Default model names on a local server

For tooling that relies on default OpenAI model names such as `gpt-3.5-turbo` , use `ollama cp` to copy an existing model name to a temporary name:

```
ollama cp llama3.2 gpt-3.5-turbo
```

Afterwards, this new model name can be specified the `model` field:

```
curl http://localhost:11434/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{
        "model": "gpt-3.5-turbo",
        "messages": [
            {
                "role": "user",
                "content": "Hello!"
            }
        ]
    }'
```

Setting the local context size

The OpenAI API does not have a way of setting the context size for a model. If you need to change the context size, create a `Modelfile` which looks like:

```
FROM <some model>
PARAMETER num_ctx <context size>
```



Use the `ollama create mymodel` command to create a new model with the updated context size. Call the API with the updated model name:

```
curl http://localhost:11434/v1/chat/completions \
    -H "Content-Type: application/json" \
    -d '{
        "model": "mymodel",
        "messages": [
            {
                "role": "user",
                "content": "Hello!"
            }
        ]
    }'
```

Previous Anthropic compatibility Next